

PatchTST: a Case Study on Electricity Forecasting

CS 5782 Deep Learning Final Project, Zhihan Liu

1 Introduction

Time-series forecasting predicts future values over a target horizon. Longer lookback windows can capture seasonal patterns, delayed dependencies, and slow trends, but point-level Transformer tokens are inefficient: individual timestamps are low-context and potentially noisy, while neighbouring timestamps are often redundant. PatchTST [1] addresses this by replacing point tokens with subseries-level patch tokens and using channel independence. Each univariate series is split into local segments, so one token represents a short temporal shape rather than a single value, and shared Transformer weights are applied independently across channels.

If the input length is L , patch length is P , and stride is S , the number of patch tokens is

$$N = \left\lfloor \frac{L - P}{S} \right\rfloor + 2 \approx \frac{L}{S}. \quad (1)$$

Thus the attention-map cost changes from roughly $O(L^2d)$ for point-level tokens to $O(N^2d)$ for patch-level tokens, with an additional patch-projection cost of $O(NPd)$. Patching therefore makes longer historical context more practical while preserving local values inside each segment.

In this project, we reproduce supervised PatchTST/64 on Electricity with prediction horizon $H = 96$, implement the paper-aligned random-masking SSL+FT setting, and add a block-masking SSL variant that masks contiguous temporal regions. We also include a scale-sensitivity probe: because patch tokens encode local temporal segments, we evaluate trained models on coarse-grained inputs to test whether the learned representation is stable under changes in temporal resolution. This diagnostic is separate from final forecasting MSE and is defined in (2) and visualized in Figure 3. In the original benchmark setting, PatchTST is compared against both Transformer-style forecasting models and simpler linear baselines such as DLinear; our reproduction focuses on whether the patch-token and channel-independent design can recover the reported gain on Electricity-96.

2 Chosen Result

Our primary chosen result is the supervised PatchTST/64 result on Electricity with prediction horizon $H = 96$, reported in Table 3 of Nie et al. as MSE 0.129 and MAE 0.222 [1]. This row is a meaningful reproduction target because Electricity is one of the paper’s large multivariate benchmarks: it contains 321 electricity-consumption channels and 26,304 timesteps, and the authors group it with Weather and Traffic as larger datasets where results are more stable and less susceptible to overfitting. It is also directly connected to PatchTST’s main design claims. Patching must preserve useful temporal structure over a long input window, while channel independence must work across many related customer-level series. The relevant architectural reference is the paper’s Figure 1, reproduced in Figure 1, which shows channel-independent patching, a shared Transformer backbone, and forecasting or reconstruction heads.

We chose the Electricity-96 setting because it is concrete, computationally feasible for a course project, and necessary for interpreting our extensions. If our supervised PatchTST/64 baseline were far from the paper’s result, then the SSL gains in Figure 2 would be hard to attribute to masked-patch pretraining rather than implementation mismatch. As a secondary reference, Table 4 of the paper reports an SSL fine-tuning result on Electricity-96 with MSE 0.126 and MAE 0.221 [1].

3 Methodology

Data. We use the UCI Electricity [2] load dataset with chronological train, validation, and test splits.

Supervised PatchTST. For the main reproduction, we use the PatchTST/64 setting summarized in Table 1:

$$L = 512, \quad H = 96, \quad P = 16, \quad S = 8.$$

This gives $N = 64$ patch tokens by (1). Each channel is instance-normalized, patchified, projected to $d = 128$, given a learned positional embedding, reshaped to $(B \cdot C) \times N \times d$, and encoded by a shared Transformer. The encoder uses 3 layers, 16 heads,

BatchNorm residual blocks, dropout 0.2, feed-forward width 512, and a flatten-linear forecasting head. The forecast is predicted in normalized coordinates and then rescaled to the original input scale. Our code and repository are available in [4].

Implementation choices. Our implementation follows the PatchTST architecture at the level of the modeling idea. The supervised PatchTST/64 reproduction uses an end-padded tokenizer: the final input value is repeated by one stride before unfolding, matching the $N = 64$ patch count for $L = 512, P = 16, S = 8$. In contrast, the paper-aligned SSL runs use a no-extra-padding tokenizer with $P = 12, S = 12$, producing 42 non-overlapping patches from the last 504 points of the 512-point input window. Thus, the supervised and SSL experiments share the same PatchTST-style encoder design and per-window instance normalization, but not the same patch granularity or padding convention.

SSL and fine-tuning. Masked SSL trains the encoder to reconstruct normalized target patches. During pretraining, selected patch tokens are replaced by a learned mask token, and a reconstruction head predicts the original normalized patch values; the SSL loss is MSE over masked patches only. After pretraining, we attach the forecasting head and fine-tune end to end on Electricity-96. Random masking is closest to the paper’s masked-patch SSL setup. Block masking is our extension: it masks contiguous temporal patch regions rather than independently selected patches, testing whether reconstructing missing time blocks gives a different forecasting representation.

Scale probe. To test the coarse-graining hypothesis, we transform the input by a temporal scale $\lambda = e^s$, resize it back to length 512, and evaluate without retraining. We report

$$r(\lambda) = \frac{\text{MSE}(f_\theta(T_\lambda x), y)}{\text{MSE}(f_\theta(x), y)}, \quad (2)$$

where T_λ is the coarse-graining-and-resizing operator. Values near 1 indicate robustness; larger values indicate degradation.

4 Results and Analysis

The supervised reproduction is the anchor result. As shown in Figure 2 and Table 1, our PatchTST/64 run obtains MSE 0.1294, compared with the paper’s MSE 0.1290. The gap is small relative to the baselines in the same figure, so we regard the reproduction as successful for the purpose of analysis. It also suggests that the essential ingredients are implemented

correctly. Remaining discrepancies could come from exact padding, scheduler details, random seed, or preprocessing conventions.

Block SSL+FT achieved the best MSE in our runs, slightly improving over Random SSL+FT even though it used a shorter fine-tuning schedule, 10 epochs rather than 15. This makes block masking a promising extension: reconstructing contiguous missing regions may encourage the encoder to learn more coherent temporal representations. However, the numerical gap is small, and the fine-tuning budgets are not identical, so we do not claim that block masking is definitively better. A fairer comparison would match the fine-tuning schedule across masking strategies or report multiple seeds with validation-based early stopping.

The scale-sensitivity experiment in Figure 3 probes a different property from benchmark MSE. Both PatchTST/64 and SSL+FT degrade as the coarse-graining level increases. The degradation is also step-like rather than smooth, suggesting that the representations learned by the model are clustered around certain characteristic scales. Taken together, the supervised lookback control, random SSL, block SSL, overlap control, and scale probe show that we did not treat reproduction as a single-number exercise; we used the reproduced baseline to test which parts of the patch-based representation seemed to matter.

5 Reflections

Our results match the the paper’s Electricity-96 result overall, suggesting a reasonable PatchTST implementation.

The SSL results were encouraging but not conclusive. Both random SSL+FT and block SSL+FT improved over our supervised PatchTST/64 run, and block masking achieved the best MSE with fewer fine-tuning epochs than random masking. This suggests that reconstructing contiguous missing regions may encourage more useful temporal representations. However, the gap is small and the fine-tuning budgets were not identical, so a stronger comparison would require matched schedules, multiple seeds, and validation-based early stopping.

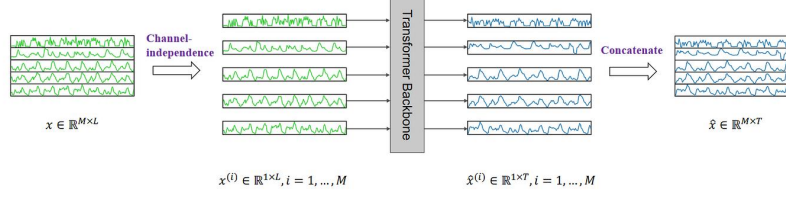
The scale-sensitivity probe asked a different question from final MSE. After coarse-graining the input, the error increased in a step-like pattern, suggesting that the learned patch representations are not scale-invariant but clustered around certain characteristic scales. Because coarse-graining and resizing may introduce artifacts, this result should be viewed as exploratory. We hope to probe the representations

learned directly and study the scale distribution of them in the future.

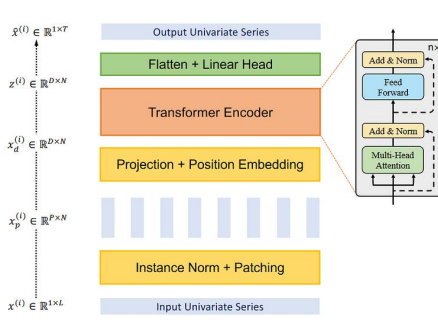
References

- [1] Y. Nie et al. *A Time Series Is Worth 64 Words: Long-term Forecasting with Transformers*. ICLR, 2023. <https://arxiv.org/abs/2211.14730>.
- [2] A. Trindade. *ElectricityLoadDiagrams20112014*. UCI Machine Learning Repository, 2015.
- [3] A. Paszke et al. *PyTorch*. NeurIPS, 2019.
- [4] Z. Liu. *PatchTST Electricity Re-implementation Repository*. GitHub, 2026. <https://github.com/KrasusZL/5782-Final-Project>.

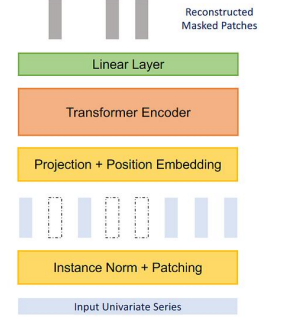
Visual Appendix



(a) PatchTST Model Overview



(b) Transformer Backbone (Supervised)



(c) Transformer Backbone (Self-supervised)

Figure 1: PatchTST architecture, reproduced from Nie et al., Figure 1 [1].

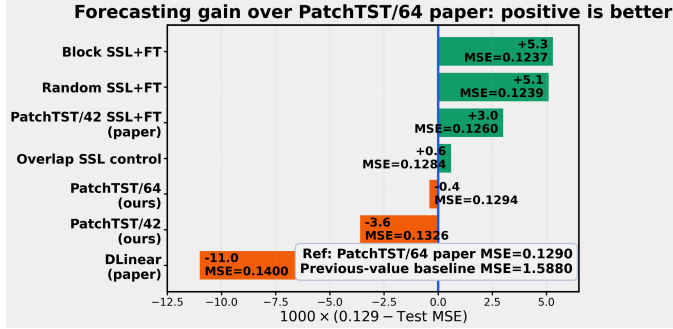


Figure 2: Project result figure. Positive gain means lower MSE than the PatchTST/64 paper baseline.

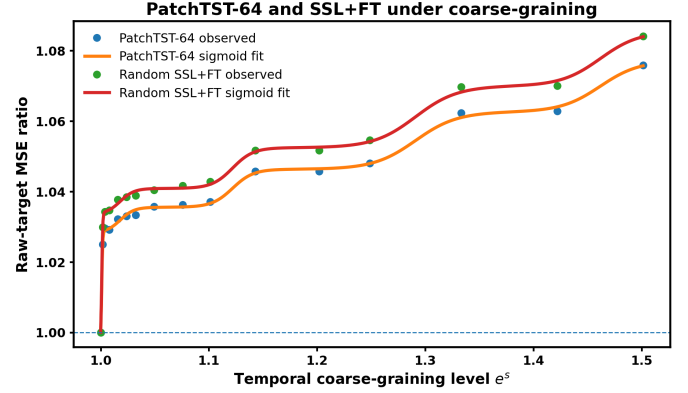


Figure 3: Scale probe. Higher raw-target MSE ratio means stronger degradation after coarse-graining.

Table 1: Complete numerical summary and experiment settings.

Method	L	H	P	S	pre/FT	Test MSE
Previous-value baseline	512	96	—	—	0/0	1.5880
DLinear paper baseline	512	96	—	—	—	0.1400
PatchTST/42 supervised	336	96	16	8	0/30	0.1326
PatchTST/64 paper reference	512	96	16	8	—	0.1290
PatchTST/64 supervised	512	96	16	8	0/30	0.1294
Overlap SSL control	336	96	16	8	30/15	0.1284
Paper SSL fine-tuning	—	96	—	—	—	0.1260
Random SSL+FT	512	96	12	12	30/15	0.1239
Block SSL+FT	512	96	12	12	30/10	0.1237